

Demo video script

A beat-by-beat script for a screen-recorded walkthrough of the AI Revenue Recovery Agent — what to say, and exactly what to click, mapped to the three dashboard tabs at `localhost:8501`.

Target runtime: **3:35** (trim Architecture beat for 3:00)

8 beats

Recovered **₹1.36cr** / **₹3.2cr** at risk (42.66%)



■ hook / close ■ problem ■ architecture ■ live dashboard demo ■ guardrails

0:00 Hook

0:15 Problem

0:40 Architecture

1:10 Overview

1:55 Risk Queue

2:25 Audit

Before you hit record

- Dashboard already running at `localhost:8501` on the **Overview** tab. Zoom the browser to ~110% so KPI numbers read clearly on a recorded screen.
- Open a second tab pre-loaded to **Audit Drill-Down** with `EVT-0016-3753a2` typed into the Event ID box — don't search for it live, that's dead air.
- Numbers in this script match the run already on disk (`data/scorecard.json`). Only re-run `python -m batch.run_batch` first if you want fresh figures on camera — not required.
- Close notification pop-ups, other tabs, chat apps. Do one silent cursor-only dry run before adding narration.

0:00–0:15

Hook

SAY

"Every payment platform loses revenue quietly — a card declines, a cart gets

SHOW

Dashboard already open on the Overview tab. Let the pipeline stepper and KPI row sit

abandoned, a subscription fails to renew, an invoice goes unpaid. This is an AI agent that catches that leakage, decides what to do about it, and proves — with real numbers — how much it recovered."

on screen for a beat before you start talking — don't click yet.

0:15–0:40

The problem

SAY

"There are four places revenue leaks: payment failures, checkout abandonment, failed subscription renewals, and overdue B2B invoices. Most teams handle all four with the same blunt tool — a generic reminder on a fixed schedule, with no sense of *why* the customer didn't pay, and no guardrail against over-contacting them."

SHOW

Point at the four rows in "Recovery by category," left to right: Payment failure, Checkout abandonment, Subscription renewal, Receivable overdue.

0:40–1:10

How it decides

SAY

"The loop is Detect, Diagnose, Decide, Guard, Act, Measure. Detect scores risk and how much is actually recoverable. Diagnose finds the root cause — plain rules first, an LLM only as fallback for messy free text. Decide is a deterministic policy table: same inputs, same intervention, every time — fully auditable. Guardrails run after that and can only make things *more* conservative — downgrade a discount, force a human escalation — never add contact the policy didn't ask for. The LLM's only job in this

SHOW

Scroll up to the pipeline stepper row under the header. Hover each of the six nodes in order as you name it.

whole loop is writing the customer-facing message."

1:10–1:55

Live demo — Overview

SAY

"This is a real batch run — 240 synthetic events, ₹3.2 crore in revenue at risk. The agent recovered ₹1.36 crore: a 42.66% recovery rate, averaging 1.3 contact steps per recovery. Receivables is the biggest lever in rupee terms; payment failures recover at the highest rate because a same-day retry works most of the time. And every event lands in exactly one bucket — recovered, escalated to a human, or guardrail-blocked. Nothing just disappears."

SHOW

Pan across the four KPI cards, then the "Recovery by category" bar chart, then the stacked "Outcome mix per category" chart, then the two reason panels at the bottom.

1:55–2:25

Live demo — Risk Queue

SAY

"Every event is ranked by priority — risk score times recoverable amount. This is what a recovery-ops person would triage from: filter by category or customer segment, and the highest-priority, highest-recoverable events float to the top."

SHOW

Click the Risk Queue tab. Toggle the category filter down to one or two categories live, then point at the sorted Priority column.

2:25–2:55

Live demo — Audit drill-down

SAY

SHOW

"This is the part judges usually ask about first: every decision is logged. Take this receivable event — Pixel Forge Studios, ₹857,966 at risk. Detect scored it, Diagnose found the root cause, Decide picked 'send invoice reminder' off the policy table, the guardrail check passed, and Act recorded the outcome: fully recovered in one step. Every stage's input and output sits right here as JSON — nothing is a black box."

Switch to the pre-loaded Audit tab (EVT-0016-3753a2). Expand the "Details" on two or three stage cards as you narrate them.

2:55–3:15

Why it's safe to trust

SAY

"The guardrails are non-negotiable. If an invoice is disputed or under a fraud hold, the agent doesn't send another message — it stops and routes straight to a human, every time. Same for anyone who's opted out. Out of this batch, 77 events were guardrail-blocked, and every block reason is logged right alongside the recoveries."

SHOW

Back on Overview, point at "Guardrail blocks" — cooldown window (50), DND opt-out (24), compliance short-circuit (13).

JUDGE-FACING POINT

This beat is where "AI agent" stops sounding risky. Say it plainly: the LLM never decides *whether* or *how much* to act — only the wording of a message that a deterministic table and a guardrail layer already approved.

3:15–3:35

Close

SAY

"Deterministic where it has to be auditable, an LLM only where it has to sound human, and a guardrail layer that

SHOW

Return to Overview, hold the frame on the "Recovery rate" KPI card as you finish the last line.

can only say no. That's the AI Revenue Recovery Agent — ₹1.36 crore recovered out of ₹3.2 crore at risk in this run, with a full audit trail behind every rupee."

COMMON PITFALLS

- **Don't wait on camera for `run_batch`.** With a live LLM key configured, Gemini's free tier caps at 5 requests/min, so a 240-event batch can run well past a minute. Pre-run it and show the finished scorecard instead.
- **Don't read every guardrail reason by name.** Point at the panel, say the count, move on — the visual is doing the work.
- **Don't narrate the tech stack.** FastAPI, Pydantic, SQLite are implementation detail; judges are scoring the loop and the audit trail, not the framework choices.
- **Move the cursor slowly and deliberately.** Fast mouse flicks are hard to follow at recorded frame rates — pause on each thing before naming it.

Numbers reflect BATCH-bd7a0429 (seed default, 240 events, live Gemini composition). A re-run will land in a similar band — the guardrail-block reasons and deterministic policy paths are what stay reproducible run to run.